AI-Powered SecOps Assistant Project Plan

https://gemini.google.com/app/71377a7dc1ad88c9

User prompt: I have to do an 8 week project at Go Reply because I started here as a graduate a month ago. It has to show my GCP skills and I will demo it at the end. Give me some ideas for the project, I like the idea of doing something AI based and/or something security based

Response: Congratulations on the new role at Go Reply! Since Go Reply is a premier Google Cloud Managed Service Provider with strong focuses on both Cloud Security and Generative AI, combining these two areas into a single 8-week project is a fantastic way to showcase your value.

An 8-week timeline is perfect for building a robust Proof of Concept (PoC) that looks visually impressive for a final demo. Here are four project ideas that blend GCP, Security, and AI:

1. The "SecOps Assistant" (AI-Powered Vulnerability Remediation)

The Concept: Security teams get overwhelmed by alerts. You can build a Slack or Google Chat bot that monitors GCP infrastructure for misconfigurations. When an alert triggers, the bot uses Vertex AI (Gemini) to explain the risk in plain English and automatically generates the exact Terraform or CLI code needed to fix it.

- **GCP Services:** Security Command Center (SCC), Vertex AI (Gemini APIs), Cloud Functions, Pub/Sub.
- **The Demo:** During your presentation, purposefully misconfigure a Cloud Storage bucket to be public. Show the audience the Slack channel instantly receiving an alert, a plain-English explanation of why it's dangerous, and a generated code snippet to lock it down.

2. Automated Phishing & PII Document Quarantine

The Concept: Build a secure document ingestion pipeline. When files (like invoices or resumes) are uploaded to a storage bucket, a serverless pipeline analyzes them. It checks for malicious links, identifies if it's a phishing attempt, and automatically redacts Personally Identifiable Information (PII) before moving it to a "safe" bucket.

- **GCP Services:** Cloud Storage, Cloud Run / Eventarc, Document AI, Cloud DLP (Data Loss Prevention), Vertex AI.
- **The Demo:** Upload a "clean" PDF and a "malicious" PDF containing fake credit card numbers and a phishing link. Show the dashboard updating in real-time as the malicious file is quarantined and the sensitive data in the clean file is completely redacted.



3. Identity Guardian: Anomaly Detection with BigQuery ML

The Concept: Insider threats and compromised credentials are a major concern. Build a system that ingests Google Cloud Audit logs and uses machine learning to establish a baseline of "normal" behavior for users, flagging anomalies (e.g., someone downloading 50GB of data at 3 AM or logging in from an unusual IP).

- **GCP Services:** Cloud Logging, BigQuery, BigQuery ML (using K-means clustering or Autoencoders for anomaly detection), Looker Studio.
- **The Demo:** Open a polished Looker Studio dashboard showing a map of normal access logs. Run a script that simulates a compromised account exfiltrating data, and watch the dashboard light up red with high-priority alerts.

4. GenAI Compliance & Runbook RAG System

The Concept: Go Reply engineers have to memorize complex compliance frameworks. Build a Retrieval-Augmented Generation (RAG) web app where engineers can ask complex questions (e.g., "How do I configure a GKE cluster to meet PCI-DSS compliance?") and get an accurate answer backed by exact citations from official documentation.

- **GCP Services:** Vertex AI Search and Conversation, Cloud Storage, LangChain, Cloud Run (for the web UI).
- **The Demo:** Show a sleek, custom-branded Go Reply web interface. Type a highly specific, complex cloud security question and watch the LLM stream a perfect architectural answer with clickable source citations.

Sources

1. Managed Services with Google Cloud | Go Reply
2. Top 50 Cyber Security Projects for Final Year Students (2026 Edition)

User prompt: i want to make the sec ops assistant, help me create an architecture diagram by explaining in more depth how it will work. (does SCC have its own AI? should i duplacate db of alerts that we manage and clean so you can opt out of repeating notifications? can I type 1,2,3 for solution options and the agent sorts it out for you? is there under settings of scc ontinuous pubsub to big query)

Response: The AI-Powered SecOps Assistant: Architecture & Setup

Building this custom workflow is an excellent way to showcase your engineering skills. While Security Command Center (SCC) does have its own built-in Gemini features in the Enterprise tier (such as case summaries and AI risk detection), building a custom event-driven pipeline using Vertex AI is much more impressive for a graduate project because it demonstrates your ability to integrate multiple GCP services and write orchestration code.

Here is the direct answer to your specific technical questions:

- **Alert Deduplication:** You should not duplicate the entire SCC database, but you **do need a lightweight state database** (Firestore is perfect for this). When an alert triggers, your code checks Firestore. If the alert ID or resource is already marked as "Ignored" or "Pending", the code stops and doesn't spam the chat.
- **Interactive Chat (1, 2, 3 Options):** Yes, this is highly encouraged. You can use Slack Block Kit or Google Chat interactive cards. When the AI generates options, you surface them as clickable buttons (e.g., *[Apply Fix]* or *[Ignore for 30 Days]*). Clicking a button sends a webhook back to GCP to execute the action.
- **SCC Continuous Exports:** Yes, SCC supports **Continuous Exports** directly to a Pub/Sub topic. You filter the export (e.g., `state="ACTIVE" AND NOT mute="MUTED"`) so only actionable findings hit your pipeline.

How It Works: Step-by-Step Architecture

This architecture relies on an event-driven serverless model. Here is the flow of data from detection to remediation.

1. Detection & Routing

- **Security Command Center (SCC):** Monitors your GCP environment. You intentionally misconfigure a resource (like making a Cloud Storage bucket public) to trigger a finding.
- **Continuous Export to Pub/Sub:** In SCC Settings, you configure a continuous export. As soon as the finding is generated, SCC pushes a JSON payload of the alert to a **Cloud Pub/Sub** topic.

2. Triage & AI Enrichment

- **Cloud Function (The Triage Engine):** A Cloud Function is subscribed to the Pub/Sub topic. When a message arrives, it triggers this function.
- **Firestore (State Management):** The function checks a **Firestore** collection (`active_alerts`). If the alert already exists and is unacknowledged, it drops the duplicate. If it's new, it creates a record.
- **Vertex AI (Gemini API):** The Cloud Function sends the raw, dense JSON alert from SCC to the Gemini API with a strict prompt: *"Translate this GCP security alert into a 2-sentence summary for a non-technical manager, and provide 3 specific gcloud CLI commands to fix, investigate, or ignore it."*

3. Human-in-the-Loop Interaction

- **Chat API (Slack or Google Chat):** The Cloud Function formats Gemini's response into an interactive message and pushes it to your security channel.
- **The Interface:** The message shows the plain-English summary, the severity, and three clickable buttons representing the options generated by Gemini.

4. Automated Execution

- **Cloud Run / API Gateway (The Executor):** When a user clicks a button in Slack (e.g., "Execute Fix"), Slack sends an HTTP POST webhook payload back to a public-facing GCP endpoint (Cloud Run is best here for security and timeout handling).
- **Action:** The Cloud Run service parses the button click, executes the associated `gcloud` command or API call to fix the bucket permissions, and updates the **Firestore** document state to "Resolved".
- **Confirmation:** It sends a final message back to the Slack thread: *"Bucket permissions restricted by [Your Name]."*

Optional Enhancements for the Demo

If you finish the core loop quickly, here are ways to make the final presentation even stronger:

- **BigQuery Analytics:** Instead of just sending findings to Pub/Sub, you can also route them to **BigQuery**. Build a quick Looker Studio dashboard that shows "Mean Time to Resolution" (MTTR) before the bot vs. after the bot.
- **Terraform Generation:** Instead of just generating `gcloud` commands, have Gemini generate the exact Terraform HCL block needed to secure the resource, allowing engineers to copy/paste it into their Infrastructure as Code repository.

Sources

1. Gemini in Security Command Center | Google Cloud Documentation
2. Overview of exporting Security Command Center data
3. Analyzing Findings with Security Command Center - Pradeep Gadde

---

User prompt: what is an impressive name for this solution

Response: Here are several impressive name ideas categorized by the "vibe" you want to project during your Go Reply demo, along with catchy taglines to use on your title slides.

Enterprise & Professional

Best for sounding like a production-ready Google Cloud product.

- **SecOps Copilot** | *Autonomous GCP Security Remediation & Intelligence*
- **SentinelAI** | *Intelligent Event-Driven Security Orchestration on GCP*
- **Aegis Response Engine (ARE)** | *Next-Gen Automated Threat Triage for Cloud Command*
- **CloudGuard Agent** | *AI-Augmented Incident Remediation for Google Cloud*

Action-Oriented & Modern

Best for showcasing speed, automation, and smart orchestration.

- **GCP SecRelay** | *Instant Incident Context & One-Click AI Remediation*
- **TriageBot 360** | *Closing the Gap Between Alert and Remediation*
- **AutoSec Operator** | *Event-Driven Security Context & Action Engine*
- **AutoShield AI** | *Autonomous Cloud Security Triage and Response*

Short & Punchy (Acronyms)

Best for quick recall during your presentation.

- **A.S.T.R.O.** | *Automated Security Triage & Remediation Orchestrator*
- **S.C.O.U.T.** | *Security Command Operations & Unification Tool*
- **S.P.A.R.K.** | *Security Policy & Automated Remediation Kernel*

Pick of the Bunch

Go with **A.S.T.R.O. (Automated Security Triage & Remediation Orchestrator)** or **SecOps Copilot**. They roll off the tongue easily, explain exactly what the tool does in five words, and fit seamlessly alongside Go Reply and GCP branding.

User prompt: I want to use SCCs built-in gemini features and generate terraform instead of gcloud commands. Create a timeline for the whole project over 8 weeks with the first 2 weeks for planning

Response: Shifting to Terraform generation and leveraging Security Command Center (SCC) Enterprise's built-in Gemini features makes this an incredibly modern and highly relevant project. SCC Enterprise uses Gemini to provide automated case management, AI-driven threat summaries, and attack path analysis.

By using SCC's native AI to understand the threat and a custom integration to translate that threat into deployable Infrastructure as Code (Terraform HCL), you are building exactly how modern DevSecOps teams want to work.

Here is your 8-week project timeline, starting with a heavy focus on planning and architecture as requested.

> Planning Phase
> • **Week 1: Requirements & Scope Definition**
>
> **Goal:** Define exactly what the project will do and which vulnerabilities it will handle.
>
> - Research SCC Enterprise's built-in Gemini capabilities (e.g., how it summarizes misconfigurations and suggests remediations).
> - Select 2-3 specific GCP misconfigurations to use for your demo (e.g., open Cloud Storage buckets, public BigQuery datasets, or overly permissive IAM roles).
> - Define the exact Terraform output required for these specific fixes.
> - Draft the user journey from the perspective of the security engineer receiving the Slack/Chat alert.

> Planning Phase
> • **Week 2: Architecture & Foundation Design**
>
> **Goal:** Finalize the technical design and provision the baseline GCP environment.
>
> - Create a detailed architecture diagram (SCC -> Pub/Sub -> Cloud Function -> Vertex AI -> Chat API -> Firestore).
> - Set up a dedicated GCP Sandbox project for your development.
> - Enable the necessary APIs: Security Command Center, Pub/Sub, Cloud Functions, Firestore, and Vertex AI.

- Design the Firestore database schema for tracking alert states (e.g., `alert_id`, `status`, `timestamp`) to prevent duplicate notifications.

Build Phase
• **Week 3: SCC Configuration & Continuous Export**

Goal: Get SCC detecting vulnerabilities and pushing them to a messaging queue.

- Configure SCC Enterprise in your sandbox environment.
- Set up a **Continuous Export** in SCC to route findings directly to a Cloud Pub/Sub topic.
- Apply filters to the export so only specific, active findings (e.g., `state="ACTIVE" AND severity="HIGH"`) trigger messages.
- Intentionally misconfigure a resource and verify the JSON payload arrives in Pub/Sub with the built-in Gemini context included.

Build Phase
• **Week 4: The Triage Engine (Cloud Function)**

Goal: Process the incoming alert and generate the Terraform code.

- Write a Python/Node.js Cloud Function triggered by the Pub/Sub topic.
- Extract the built-in Gemini threat summary and remediation steps from the SCC JSON payload.
- Pass those remediation steps into the Vertex AI API (Gemini Pro) with a strict system prompt: *"You are a DevSecOps engineer. Translate these remediation steps into valid Terraform (HCL) code. Output ONLY the Terraform block."*

Build Phase
• **Week 5: Chat Integration & UI**

Goal: Surface the alert and the Terraform code to the user.

- Create a Slack App or Google Chat webhook integration.
- Format the Cloud Function output into an interactive UI card. The card should display:
    1. The AI-generated threat summary from SCC.
    2. A code block containing the generated Terraform code.
    3. Interactive buttons (e.g., *[Acknowledge]*, *[Mark as False Positive]*).

Build Phase
• **Week 6: State Management & Deduplication**

Goal: Ensure the bot isn't spammy and handles state gracefully.

- Integrate Firestore into your Cloud Function.
- Write logic to check if an `alert_id` already exists in Firestore before sending a chat message.
- Build a webhook receiver (API Gateway or Cloud Run) to handle button clicks from the chat interface. When a user clicks *[Acknowledge]*, update the Firestore document so the alert is silenced.

Testing Phase
• **Week 7: End-to-End Testing & Refinement**

Goal: Break it, fix it, and optimize the AI prompts.

- Trigger multiple different misconfigurations and monitor the end-to-end flow.
- Review the generated Terraform. Is it accurate? Does it use the correct provider syntax? Tweak the Vertex AI prompt to improve the HCL formatting.
- Test edge cases (e.g., what happens if SCC triggers the same alert twice in one minute?).

Presentation Phase
• **Week 8: Demo Preparation & Polish**

Goal: Ensure a flawless presentation for Go Reply.

- Prepare your demo script. Focus on the business value: saving time and enforcing Infrastructure as Code best practices.
- Create a short slide deck summarizing the architecture and the GCP services used.
- **Crucial:** Record a video of the system working flawlessly. Live demos can break due to network issues or API delays; having a backup video guarantees success.
- Practice the live run-through.

Sources

1. Security Command Center | Google Cloud

User prompt: Help me draft the user journey from the perspective of the security engineer receiving the google chat alert

Response: Here is the user journey mapped from the perspective of a Go Reply DevSecOps engineer.

This journey highlights the shift from a traditional, reactive workflow (digging through logs to figure out what happened) to a modern, proactive workflow (reviewing AI-generated context and applying infrastructure-as-code fixes instantly).

1. **The Notification**

   High signal, low noise

   The engineer is working on another task when a notification pings in the `#gcp-secops-alerts` Google Chat space. Instead of a noisy, raw JSON dump, they see a neatly formatted, color-coded interactive card tagged **[HIGH SEVERITY]**.

2. **Instant Comprehension**

   Powered by SCC Enterprise Gemini

   The engineer reads the "Threat Summary" section of the card. Because SCC's built-in AI has already analyzed the event, they instantly read a plain-English explanation: *"The Cloud Storage bucket `prod-customer-data` was modified 2 minutes ago to allow `allUsers` read access, exposing potentially sensitive data to the public internet."* They understand the risk immediately without needing to open the GCP Console.

3. **Reviewing the IaC Fix**

   Powered by Vertex AI

   Directly below the summary, the engineer sees a "Remediation" section containing a pre-written Terraform block. They review the HCL code and confirm it correctly re-establishes the private IAM bindings for the bucket.

4. **Taking Action**

   Interactive Chat Elements

   At the bottom of the card, the engineer clicks an interactive button labeled **[Acknowledge & Claim]**. The Google Chat card instantly updates to show the rest of the team that this specific engineer is handling the incident. Behind the scenes, your Cloud Function updates the Firestore database to silence duplicate alerts for this specific bucket.

5. **Deployment & Resolution**

   Enforcing GitOps best practices

   The engineer clicks a **[Copy Code]** button, pastes the generated Terraform into their organization's infrastructure repository, and opens a quick Pull Request. Once approved and merged, the CI/CD pipeline applies the fix. The incident is resolved entirely through approved Infrastructure as Code workflows in a fraction of the usual time.

User prompt: could the agent have permission to deploy the terraform itself

Response: **Yes, technically it can.** However, in a professional production environment—especially in a Google Cloud Managed Service Provider like Go Reply—giving an automated bot direct permissions to unilaterally modify or apply Terraform to production infrastructure introduces significant security and operational risks.

Here is an analysis of how to safely handle automated deployment, along with the architecture that bridges the gap between full automation and governance.

Why Direct Execution Has Risks

1. **Privilege Escalation & Blast Radius:** To run `terraform apply`, the bot's service account would need broad administrative permissions (e.g., `roles/storage.admin`, `roles/resourcemanager.organizationAdmin`). If an attacker manipulates an alert or triggers a prompt injection attack, they could force the bot to execute unintended infrastructure changes.
2. **State File Drift:** If an engineer manages infrastructure via Terraform (GitOps), having an external bot modify resources out-of-band directly on GCP causes **state drift**. The next time an engineer runs `terraform plan`, their local environment will attempt to revert or override the bot's changes.
3. **Lack of Peer Review:** Enterprise compliance (e.g., ISO 27001, SOC 2, PCI-DSS) strictly requires a "four-eyes principle" (dual-authorization) for production infrastructure changes.

The Recommended Solution: Automated Pull Requests (GitOps)

Instead of the bot running `terraform apply` directly against GCP, the bot acts as a developer's assistant:

1. **The Click:** The engineer reviews the alert in Google Chat and clicks **[Apply Fix via PR]**.
2. **The Automation:** The backend Cloud Function calls the GitHub or GitLab API using a dedicated bot token.
3. **The Pull Request:** The bot automatically clones the infrastructure repository, creates a new branch, adds the generated Terraform block, and opens a Pull Request.
4. **The Review:** The engineer inspects the PR diff, approves it, and merges it.
5. **The Deployment:** The existing CI/CD pipeline (e.g., Cloud Build or GitHub Actions) runs `terraform apply` safely under controlled deployment credentials.

Comparison of Deployment Models

Feature	Option A: Direct Bot Execution	Option B: Automated PR (Recommended)
Speed to Fix	Instant (seconds)	Fast (1–2 minutes)
State Drift Risk	High (modifies GCP without updating `.tfstate`)	Zero (keeps Git repo and GCP state synchronized)
Security Risk	High (requires broad admin service account keys)	Low (only requires scoped Git repo write access)
Audit Compliance	Difficult (hard to track who authorized the change)	Native (tracked via Git commit history and PR approvals)

How to Highlight This in Your Demo

For your Go Reply demo, explaining *why* you chose a GitOps workflow over direct execution is a strong demonstration of senior cloud architecture thinking.

- **During the presentation:** Explicitly state: *"While we could give the Cloud Function permission to run `terraform apply` directly using a Service Account, enterprise security best practices require keeping Git as the single source of truth. Therefore, the bot automatically creates a Pull Request rather than modifying production out-of-band."*
- **Alternative for non-production environments:** If you do want to show end-to-end automated deployment for demo purposes, restrict the bot's scope to a dedicated **Sandbox/Dev GCP Project** with a strictly isolated Terraform state file.

User prompt: help me Design the Firestore database schema for tracking alert states (e.g., alert_id, status, timestamp) to prevent duplicate notifications.

Response: To prevent duplicate notifications and maintain state, your Firestore database should use a single collection (e.g., `secops_alerts`). The most critical design decision is to use the **SCC Finding ID as the Firestore Document ID**, which allows your Cloud Function to perform an instant, cost-effective O(1) lookup to check if an alert has already been processed.

Firestore Schema (`secops_alerts` collection)

Field Name	Data Type	Description
`[Document ID]`	String	The unique SCC finding name (e.g., `organizations/123/.../findings/abc`).
`status`	String	Current state: `NOTIFIED`, `ACKNOWLEDGED`, `PR_OPEN`, `RESOLVED`, `IGNORED`.
`resource_name`	String	The GCP resource affected (e.g., `projects/my-proj/buckets/data`).
`category`	String	The type of finding (e.g., `IAM_ANOMALY`, `PUBLIC_BUCKET`).
`severity`	String	`CRITICAL`, `HIGH`, `MEDIUM`, `LOW`.
`created_at`	Timestamp	When the finding was first ingested into Firestore.
`updated_at`	Timestamp	When the status was last changed.
`chat_message_id`	String	The ID of the Google Chat message (used to update the card later).
`handled_by`	String	(Optional) The email of the engineer who clicked "Acknowledge" in Chat.

Example Document Payload

When you view the document in the Firebase Console, it will look like this:

```
{
  "status": "NOTIFIED",
  "resource_name": "//storage.googleapis.com/prod-customer-data",
  "category": "PUBLIC_BUCKET_IAM",
  "severity": "HIGH",
  "created_at": "2026-10-12T15:30:00Z",
  "updated_at": "2026-10-12T15:30:05Z",
  "chat_message_id": "spaces/AAAABBBCCC/messages/123456",
  "handled_by": ""
}
```

The Deduplication Logic (Cloud Function Flow)

When your Cloud Function receives a Pub/Sub message from SCC, it should execute the following logic to ensure the chat channel remains quiet:

1. **Extract the ID:** Parse the incoming JSON to get the unique `finding.name`.
2. **Check Firestore:** Perform a `get()` request on the `secops_alerts` collection using that ID.
3. **Evaluate State:**

- **If the document DOES NOT exist:** This is a net-new alert. Send the message to Google Chat, get the `chat_message_id` back from the Chat API, and save the new document to Firestore with `status: NOTIFIED`.
- **If the document EXISTS and status is `NOTIFIED`:** The bot already sent a message, but no engineer has claimed it yet. **Do nothing** (drop the message).
- **If the document EXISTS and status is `ACKNOWLEDGED` or `PR_OPEN`:** An engineer is actively working on it. **Do nothing**.
- **If the document EXISTS and status is `RESOLVED`:** The issue was previously fixed but SCC has detected it *again*. This represents a regression. Update the status back to `NOTIFIED`, and send a new message to chat stating: *"Regression Detected: This vulnerability was previously resolved but has reopened."*

User prompt: using firestore vs using bigquery for this

Response: Using Firestore instead of Bigquery is the right approach for this specific problem. While BigQuery is an incredibly powerful data warehouse for analytics, it is the wrong tool for the high-speed, transactional state-tracking required by a live SecOps chatbot.

Here is why Firestore is the correct choice, and how it compares to BigQuery for this architecture:

Feature	Firestore (Recommended)	BigQuery
Primary Use Case	Real-time state management, transactional deduplication, and fast lookups.	Large-scale data warehousing, analytics, and complex aggregations.
Lookup Latency	Single-digit milliseconds (O(1) key-value lookup).	Seconds to minutes (designed for massive table scans, not point reads).
Deduplication Speed	The Cloud Function can check if an alert exists instantly before sending a Slack/Chat message.	Querying BigQuery to check state would be too slow, potentially causing duplicate chat messages.
Pricing Model	Pay per document read/write (fractions of a cent).	Pay per TB of data scanned. Checking state would scan large tables and incur unnecessary costs.
Updates	Easily updates a single document's status (e.g., `NOTIFIED` to `RESOLVED`).	BigQuery is append-mostly. Updating single rows frequently is an anti-pattern.

When *Should* You Use BigQuery?

You shouldn't replace Firestore with BigQuery, but you **can use them together** to make your demo even more impressive.

This is called a **Lambda Architecture** (handling both real-time streams and long-term analytics):

1. **Firestore (The Operational Database):** Manages the real-time state of the alert (Has the Slack message been sent? Has someone clicked "Acknowledge"?).
2. **BigQuery (The Analytical Database):** In the background, SCC also streams all alerts into BigQuery. You use this to build a Looker Studio dashboard showing historical trends, such as:
   - "How many Public Buckets did we create this month?"
   - "What is our Mean Time to Resolution (MTTR) since we launched the bot?"

For the core deduplication logic of your bot, stick to Firestore.

Sources

1. Google Bigtable vs Firestore: Key Differences Explained - NoCodeAPI

User prompt: Lets revise the architecture diagram. Using SCC inbuilt ai to generate fix suggestions, scc continuous pubsub to bigquery and firestore, a duplicate db of alerts that we manage and clean so you can opt out of repeating notifications, receiving a summary and fix options in google chat, ai generates a terraform fix and makes a pull request, a confirmation message is sent to the chat

Response: This is an excellent evolution of the architecture. By running SCC directly to BigQuery for analytics and using Firestore for the operational state, you are implementing a true Lambda Architecture (handling both the real-time event stream and long-term reporting).

Here is the revised step-by-step breakdown of your architecture, followed by an interactive visualization of the flow.

1. Detection & Fan-Out Routing

- **Security Command Center (SCC):** Detects the misconfiguration (e.g., an open Cloud Storage bucket) and uses its built-in Gemini capabilities to analyze the threat, generate a plain-English summary, and define remediation steps.
- **Continuous Export (Pub/Sub):** SCC continuously pushes these enriched JSON payloads to a single Cloud Pub/Sub topic.
- **The Fan-Out:** The Pub/Sub topic has two separate subscriptions:
  - **Subscription A (Analytics):** Routes the payload directly to **BigQuery** (via Dataflow or a direct BigQuery subscription) for historical tracking, MTTR calculations, and Looker Studio dashboards.

- **Subscription B (Operations):** Triggers your custom **Cloud Function** (The Triage Engine).

2. Deduplication & State Management

- **Cloud Function (The Triage Engine):** Receives the event and instantly checks your **Firestore** database (`secops_alerts` collection) using the SCC Finding ID as the document key.
- **Opt-Out/Deduplication Logic:**
  - If the Firestore document status is `IGNORED` (the user previously opted out of this specific alert), `NOTIFIED`, or `PR_OPEN`, the function drops the event to prevent spam.
  - If it is a new alert, it logs it as `NOTIFIED` and proceeds.

3. Chat Notification & Triage

- **Google Chat API:** The Cloud Function formats the SCC Gemini summary and remediation options into an interactive Google Chat card.
- **The Interface:** The security engineer sees the threat summary in their Chat Space and is presented with interactive buttons: *[Acknowledge]*, *[Ignore]* or *[Generate Terraform Fix]*.

4. AI-Driven Remediation & GitOps

- **Cloud Run (Webhook Receiver):** When the engineer clicks *[Generate Terraform Fix]*, the button sends an HTTP POST to your Cloud Run endpoint.
- **Vertex AI Generation:** Cloud Run takes the remediation instructions from the original SCC payload and sends them to Vertex AI with a strict prompt to output valid Terraform HCL.
- **Automated PR Creation:** Cloud Run uses a GitHub/GitLab bot token to clone your infrastructure repository, inject the generated `.tf` code, and open a Pull Request. It then updates the Firestore state to `PR_OPEN`.

5. Resolution & Confirmation

- **Chat Update:** Cloud Run sends an asynchronous confirmation message back to the Google Chat thread: *"✅ Terraform Fix Generated. Review the PR here: [GitHub Link]"*.
- **GitOps Flow:** The engineer reviews the PR, merges it, and the CI/CD pipeline applies the fix securely to the GCP environment.

User prompt: Update the Gantt chart milestones to fit the new project outline

Response: Here is your updated 8-week timeline, adjusted to perfectly match the revised architecture including the BigQuery analytics fan-out, the interactive Chat App, Vertex AI Terraform generation, and the automated Pull Request GitOps flow.

> Planning Phase
> • **Week 1: Requirements & Use Case Definition**
>
> **Goal:** Define the exact vulnerabilities and GitOps repository structure.
>
> - Research SCC Enterprise's built-in Gemini payload structure.
> - Select 2-3 specific GCP misconfigurations to use for the demo.
> - Set up a dummy GitHub/GitLab repository to act as your "infrastructure-as-code" target for the Pull Requests.

> Planning Phase
> • **Week 2: Architecture & Sandbox Setup**
>
> **Goal:** Finalize the design and provision the baseline GCP environment.
>
> - Enable the necessary APIs: SCC, Pub/Sub, Cloud Functions, Cloud Run, Firestore, BigQuery, and Vertex AI.
> - Design the Firestore schema for state management and deduplication.
> - Create the BigQuery dataset and table schema for long-term analytics.

> Build Phase
> • **Week 3: SCC Export & The Data Fan-Out**
>
> **Goal:** Configure the detection engine and route the data to both storage systems.
>
> - Set up SCC Continuous Export to your primary Pub/Sub topic.
> - **Analytics Route:** Create a Pub/Sub subscription that streams events directly into BigQuery.
> - **Operations Route:** Create a Pub/Sub subscription to trigger your initial Cloud Function.

> Build Phase
> • **Week 4: Deduplication & Google Chat UI**
>
> **Goal:** Process the alert, prevent spam, and surface the interactive card.
>
> - Write the Cloud Function to check Firestore. If the alert is new, log it as `NOTIFIED`; if it exists, drop it.

- Integrate the Google Chat API. Send a formatted UI card containing the SCC built-in threat summary and interactive buttons (e.g., *[Acknowledge]*, *[Generate PR]*).

Build Phase
• **Week 5: Vertex AI & Terraform Generation**

Goal: Handle the engineer's button click and generate the code.

- Deploy a Cloud Run service to act as the webhook receiver for Google Chat button clicks.
- When *[Generate PR]* is clicked, extract the remediation steps from the SCC payload.
- Send the steps to the Vertex AI API (Gemini Pro) to generate the exact Terraform HCL block.

Build Phase
• **Week 6: GitOps PR Automation & Confirmation**

Goal: Push the generated code to Git and close the loop with the engineer.

- Add Git API logic to your Cloud Run service to branch the repo, commit the Terraform block, and open a Pull Request.
- Update the Firestore document status to `PR_OPEN`.
- Send an asynchronous confirmation message back to the Google Chat thread with the PR link.

Testing Phase
• **Week 7: Analytics Dashboard & E2E Testing**

Goal: Build the visual metrics and test the entire pipeline.

- Connect **Looker Studio** to your BigQuery dataset to build a quick dashboard showing "Total Vulnerabilities Detected" and "Mean Time to Action".
- Trigger your test vulnerabilities in GCP and watch the full flow from detection to Pull Request creation.

Presentation Phase
• **Week 8: Demo Preparation & Polish**

Goal: Ensure a flawless presentation for Go Reply.

- Prepare the demo script emphasizing the business value: reducing alert fatigue with deduplication, and enforcing GitOps best practices.
- Record a backup video of the system working end-to-end to mitigate live-demo risks.

User prompt: Hi Anthony! You mentioned in our first chat about the capstone project that SCC can support a continuous pub/sub export to bigquery. You also mentioned a duplicate DB of alerts to allow opting out of recurring notifications. To me it seems that firestore would be best for this db and I dont see why bigquery would be useful other than maybe for an analytics dashboard. What do you think? Is there a use for bigquery here that im not thinking of? Anthony Youssef, Yesterday 14:59 Heyyyy We definitely need Firestore as our operational state database to handle the live chat sessions, fast deduplication, and the opt-out/ignore list—BigQuery is too slow for transactional chat state. However, setting up the BigQuery export alongside it is a great idea because it takes 5 minutes to configure via Pub/Sub, gives us an immutable audit trail, and lets our agent query historical trends or join with Audit Logs before messaging the security team. So basically, I think you should have both ... Now the real challenge is determining what the BigQuery schema should look like, and based on that, identifying the optimal Firestore schema to complement it. You, Yesterday 15:21 Got it, thank you! Should their schemas be the same or slightly different? Anthony Youssef, Yesterday 15:27, Edited No diff, you don't want to duplicate data that you don't need to Then you are paying for more storage and also maintaining the data in two places and making sure they are synced in both... becomes messy generate the schemas for firestore and bigquery based on anthony's advice

Response: The user is asking for the schemas for Firestore and BigQuery to track Security Command Center (SCC) alerts based on a discussion with Anthony Youssef. Anthony says: "No diff, you don't want to duplicate data that you don't need to" and "generate the schemas for firestore and bigquery based on anthony's advice".

Wait, if there's no difference, the schemas should be exactly the same (except maybe for data type mapping between NoSQL and SQL). I should generate identical schemas.

What are the fields I need? From my previous response and SCC finding structure: `finding_name` (String) - ID `status` (String) `resource_name` (String) `category` (String) `severity` (String) `created_at` (Timestamp) `updated_at` (Timestamp) `chat_message_id` (String) `handled_by` (String)

I'll provide the BigQuery table schema (DDL) and the Firestore document structure (JSON/TypeScript interface), showing that they are mapped 1:1 as Anthony advised.

Let's output the schema.

BigQuery Schema (Standard SQL DDL):

```
CREATE TABLE `your-project.scc_analytics.alerts_state` (
  finding_name STRING NOT NULL OPTIONS(description="The unique SCC finding name (e.g., organizations/123/s
  status STRING NOT NULL OPTIONS(description="Current state: NOTIFIED, ACKNOWLEDGED, PR_OPEN, RESOLVED, IG
  resource_name STRING OPTIONS(description="The GCP resource affected"),
  category STRING OPTIONS(description="The type of finding"),
  severity STRING OPTIONS(description="CRITICAL, HIGH, MEDIUM, LOW"),
  created_at TIMESTAMP OPTIONS(description="When the finding was first ingested"),
  updated_at TIMESTAMP OPTIONS(description="When the status was last changed"),
  chat_message_id STRING OPTIONS(description="The ID of the Google Chat message"),
```

```
  handled_by STRING OPTIONS(description="The email of the engineer who clicked Acknowledge")
)
```

Firestore Schema: Document ID: `finding_name` (e.g., `organizations/123/sources/456/findings/abc` with slashes replaced or handled correctly, or just use the finding ID part). Fields:

- `status`: String
- `resource_name`: String
- `category`: String
- `severity`: String
- `created_at`: Timestamp
- `updated_at`: Timestamp
- `chat_message_id`: String
- `handled_by`: String

I will explain that to keep the data synced and avoid messiness, we use a single Cloud Function to update both when a status changes (e.g. from NOTIFIED to PR_OPEN).

Let's format the response.

- Start directly.
- Show both schemas.
- Make sure they mirror each other.

Wait, since Pub/Sub can export directly to BigQuery, does it use the raw SCC schema or this custom state schema? Anthony said "determining what the BigQuery schema should look like, and based on that, identifying the optimal Firestore schema to complement it" and then "No diff, you don't want to duplicate data that you don't need to". This implies we use the same custom operational schema for BigQuery as we do for Firestore.

I will write the response directly.Anthony is absolutely right—keeping the schemas identical between Firestore and BigQuery is the cleanest approach. It reduces cognitive load, simplifies your Cloud Function logic, and ensures your Looker Studio dashboards accurately reflect the exact state managed by the Chatbot.

Here are the identical schemas translated into their respective database formats.

1. BigQuery Schema (Analytical Audit Trail)

You can create this table in BigQuery using the following Standard SQL Data Definition Language (DDL) statement. This table serves as your immutable audit log.

```
CREATE TABLE `your-project.scc_analytics.alerts_state` (
  finding_name STRING NOT NULL OPTIONS(description="The unique SCC finding name (Document ID in Firestore)
  status STRING NOT NULL OPTIONS(description="Current state: NOTIFIED, ACKNOWLEDGED, PR_OPEN, RESOLVED, IG
  resource_name STRING OPTIONS(description="The GCP resource affected (e.g., projects/my-proj/buckets/data
  category STRING OPTIONS(description="The type of finding (e.g., PUBLIC_BUCKET_IAM)"),
  severity STRING OPTIONS(description="CRITICAL, HIGH, MEDIUM, LOW"),
  created_at TIMESTAMP OPTIONS(description="When the finding was first ingested"),
  updated_at TIMESTAMP OPTIONS(description="When the status was last changed"),
  chat_message_id STRING OPTIONS(description="The ID of the Google Chat message"),
  handled_by STRING OPTIONS(description="The email of the engineer who took action")
)
PARTITION BY DATE(created_at);
```

(Note: Partitioning by `created_at` is a GCP best practice that makes long-term storage cheaper and queries faster.)

2. Firestore Schema (Operational State)

In Firestore, your collection (e.g., `secops_alerts`) will use the exact same fields. The only difference is that the `finding_name` is used as the Document ID itself for those lightning-fast $O(1)$ lookups.

Here is the JSON representation of a document in Firestore:

```
{
  // Document ID: organizations/123/sources/456/findings/abc
  "status": "PR_OPEN",
  "resource_name": "//storage.googleapis.com/prod-customer-data",
  "category": "PUBLIC_BUCKET_IAM",
  "severity": "HIGH",
  "created_at": "2026-10-14T15:30:00Z",
  "updated_at": "2026-10-15T09:12:00Z",
  "chat_message_id": "spaces/AAAABBBCCC/messages/123456",
  "handled_by": "anthony.youssef@goreply.com"
}
```

How to Keep Them Synced Without the Mess

Since Anthony highlighted the risk of messy synchronization, here is how you handle it cleanly in your Cloud Function:

1. **The Initial Ingestion:** When SCC pushes a new alert to Pub/Sub, your Cloud Function triggers. It writes the document to Firestore (`status: NOTIFIED`). In the **exact same function execution**, it inserts an identical row into BigQuery.
2. **The Status Update:** When an engineer clicks **[Generate PR]** in Google Chat, your Cloud Run service updates the Firestore document to `status: PR_OPEN`. It immediately inserts a *new* row into BigQuery with the same `finding_name` but the new `status` and `updated_at` timestamp.

By appending rows in BigQuery rather than updating them, BigQuery acts as a perfect historical ledger of every state change, while Firestore always holds the single "current" state.

User prompt: Write a brief project description that I will send to my supervisor for approval

Response: Here is a concise, executive-ready project proposal designed for supervisor approval:

Project Proposal: Automated SecOps Triage & GitOps Remediation Engine

Objective:

To design and deploy an event-driven, AI-augmented security remediation pipeline that reduces alert fatigue, accelerates incident response, and enforces Infrastructure-as-Code (IaC) best practices across Google Cloud infrastructure.

Project Overview:

When Google Cloud Security Command Center (SCC) Enterprise detects a high-priority misconfiguration (e.g., an exposed Cloud Storage bucket), an automated pipeline ingests the alert and streams data simultaneously into two channels:

1. **Analytical Audit Layer (BigQuery):** An immutable event ledger for compliance auditing, historical reporting, and Looker Studio MTTR metrics.
2. **Operational State Layer (Firestore):** A fast, key-value lookup database that manages real-time alert states (`NOTIFIED`, `PR_OPEN`, `RESOLVED`, `IGNORED`) to prevent duplicate chat notifications and allow team members to opt out of recurring alerts.

Upon receiving a new alert, a serverless Cloud Function leverages SCC's built-in Gemini threat summaries and triggers Vertex AI (Gemini) to convert the remediation guidance directly into deployable Terraform (HCL) code. The alert and proposed Terraform fix are sent to Google Chat as an interactive card.

Rather than allowing the bot to modify production infrastructure directly, clicking **[Generate PR]** triggers a Cloud Run service that automatically opens a Pull Request in GitHub/GitLab—keeping Git as the single source of truth and enforcing code review before deployment.

Key GCP Technologies:

Security Command Center Enterprise, Cloud Pub/Sub, Cloud Functions, Cloud Run, Firestore, BigQuery, Vertex AI, and Google Chat API.

Business & Technical Value:

- **Drastically Reduced MTTR:** Eliminates manual log analysis by delivering instant, AI-generated context and ready-to-deploy fixes to security engineers.
- **GitOps Compliance:** Mitigates risk by generating Pull Requests rather than performing out-of-band infrastructure modifications.
- **Operational Efficiency:** Deduplicates redundant alerts and tracks operational state in real-time.

User prompt: Make it a three sentence summary

Response: This eight-week project delivers an AI-powered security remediation pipeline that captures Security Command Center alerts, logs them to BigQuery for compliance auditing, and uses Firestore to seamlessly deduplicate notifications. When a vulnerability is detected, Vertex AI automatically translates the threat into deployable Terraform code and presents it to engineers via an interactive Google Chat alert. By allowing engineers to resolve the issue with one click via an automated GitOps Pull Request, this solution drastically reduces response times while enforcing strict infrastructure-as-code controls.

User prompt: Take the best bits from each of these: This project delivers an AI-powered security remediation pipeline that captures Security Command Center alerts, logs them to BigQuery for compliance auditing, and uses Firestore to seamlessly deduplicate notifications. When a vulnerability is detected, Vertex AI automatically translates the threat into deployable Terraform code and presents it to engineers via an interactive Google Chat alert. By allowing engineers to resolve the issue with one click via an automated GitOps Pull Request, this solution drastically reduces response times while enforcing strict infrastructure-as-code controls. CloudGuard Agent is an autonomous cloud security assistant that turns Google Cloud Security Command Center (SCC) findings into instant, actionable decisions inside Google Chat. With Gemini analysing the threat, the agent drafts the well-thought suggestions and possible remediations. CloudGuard bridges threat detection and remediation from minutes to a single tap. Whether you need to deploy an automated Terraform fix via GitOps, assign a GitHub issue, or escalate to an owner, CloudGuard lets security and DevOps teams remediate risks in seconds directly from chat.

Response: **CloudGuard Agent** is an autonomous cloud security assistant that turns Google Cloud Security Command Center (SCC) findings into instant, actionable decisions directly inside Google Chat. Powered by Vertex AI, the pipeline seamlessly deduplicates alerts using Firestore, logs an audit trail to BigQuery, and automatically translates threats into deployable Terraform code. By allowing teams to resolve issues with a single tap via an automated GitOps Pull Request, CloudGuard accelerates incident remediation from minutes to seconds while enforcing strict infrastructure-as-code controls.

User prompt: I need to know the full capabilities of SCC and its inbuilt AI, Anthony thinks that I need a vertex AI to clean SCC logs to a more suitable format before storing them in firestore, is this accurate or can the info be exported from SCC in a format which is already well structured and can be cleaned without AI

Response: Anthony is incorrect about needing Vertex AI to clean the logs. Security Command Center (SCC) natively exports findings to Pub/Sub as highly structured, predictable JSON payloads. Using a Large Language Model to parse or "clean" strict JSON is an architectural anti-pattern—it introduces unnecessary latency, API costs, and the risk of hallucination into a simple Extract-Transform-Load (ETL) process.

You can reliably extract the necessary fields using standard Python or Node.js dictionary parsing in your Cloud Function before writing the state to Firestore.

SCC Core Capabilities

SCC is Google Cloud's native security posture management and threat detection platform. Its primary capabilities include:

- **Asset Discovery:** Continuously scanning GCP environments for resources and IAM policies.
- **Threat Detection & Vulnerability Management:** Identifying misconfigurations (e.g., public storage buckets) and active threats (e.g., anomalous IAM grants, malware).
- **Continuous Export:** Streaming findings in real-time to Pub/Sub or BigQuery using standard SQL-like filtering (e.g., `state="ACTIVE" AND severity="HIGH"`) so you only process actionable alerts.

SCC Enterprise Inbuilt AI (Gemini)

In the Enterprise tier, SCC integrates Gemini directly into the threat detection engine. The AI does the heavy lifting *before* the log is ever exported. Its built-in capabilities include:

- **Plain-English Summaries:** Translating complex cloud alerts into human-readable incident descriptions.
- **Attack Path Analysis:** Identifying how an attacker could move laterally through your environment.
- **Remediation Guidance:** Providing step-by-step instructions or CLI commands to fix the vulnerability natively within the payload.

How to Process the Log Without AI

When SCC pushes an alert to Pub/Sub, the message contains a strict JSON schema. Your Cloud Function simply maps the native JSON keys directly to your Firestore database schema:

- `finding.name` → Maps to your Firestore Document ID.
- `finding.category` → The type of vulnerability (e.g., `PUBLIC_BUCKET_IAM`).
- `finding.state` → The status of the vulnerability in GCP (e.g., `ACTIVE`).
- `finding.severity` → The urgency (e.g., `HIGH`, `CRITICAL`).

The correct use of Vertex AI in your project: Keep Vertex AI out of the data ingestion and cleaning phase. Instead, extract SCC's built-in AI remediation text from the JSON using standard code, and pass *only* that text to Vertex AI when the user clicks the chat button to generate the custom Terraform HCL block.

Sources

1. Creating and managing Notification Configs | Security Command
2. Analyzing Findings with Security Command Center - Pradeep Gadde

---